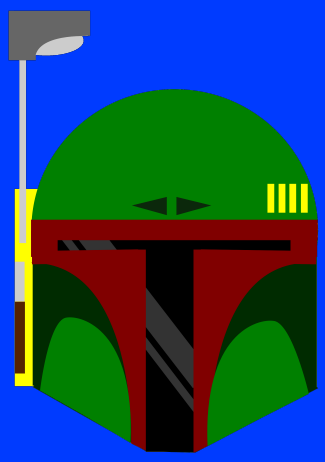
{EPITECH}

RTC STRIKES BACK_

< UPGRADE YOUR APPLICATION />



RTC STRIKES BACK

Context

You have developed the foundations of a chat application with a solid Rust or NodeJS backend. It's now time to evolve this product into a complete and professional application.
As an investor, I have specific expectations regarding the features to deliver and the quality of the final product.

This project aims to simulate real startup conditions:

- ✓ Tight time constraints;
- ✓ Team to manage;
- ✓ Technical choices to make and assume;
- ✓ Delivery of a functional product;
- ✓ Presentation to investors/clients.

Objectives

You must evolve your RTC through 3 **successive phases**:

> ⚠️ Each phase should be completed before diving into the next one

Phase 1: Completion of Core Features

Your application must allow complete management of servers and messages.

Expected features:

- ✓ **Kick**: Expel a member from a server (they can return);
- ✓ **Permanent Ban**: Ban a member permanently from a server;
- ✓ **Temporary Ban**: Ban a member with an expiration duration;
- ✓ **Message Editing**: Allow users to edit their own messages.

Phase 2: Enhancement and Professionalization

Your application must become richer and demonstrate professional practices.

Expected features:

- ✓ **internationalization**: Multilingual interface (at least FR/EN) in the client application;
- ✓ **CI/CD**: Continuous Integration & Continuous Deployment pipeline on GitHub;
- ✓ **External GIF API**: Integration of a GIF API (Tenor, Giphy, etc.);
- ✓ **Private Message**: Direct messaging system between users;
- ✓ **Message Reactions**: Emoji reaction system on messages.

> Your CI/CD should work when you push on a branch or when you merge your branch on the main. You should also be able to build your app automatically when a tag is created.

Phase 3: Desktop Application

Your backend must be accessible through a native desktop application built with Tauri or Electron.

Expected features:

- ✓ Installable desktop application (Windows, macOS, Linux);
- ✓ Complete graphical interface allowing use of all the features;
- ✓ Multilingual (at least FR/EN);
- ✓ System notifications.

Technology requirement: The desktop application must be built using either:

- ✓ **Tauri**;
- ✓ **Electron**

Keynote

You have to:

- ✓ Present your project in front of an audience;
- ✓ Make a live demonstration;
- ✓ Answer technical and organizational questions.

The expected structure of your keynote is:

- ✓ Introduction;
- ✓ Technical Architecture;
- ✓ Methodology;
- ✓ Live Demonstration;
- ✓ Code walkthrough of 1 feature.

⚠️ Every team member must be able to answer any question, whether technical or organizational.

Specific Expectations

What We Want to See

- ✓ **A product that works** - Better to have fewer features that work than many that crash
- ✓ **Professionalism** - Clean code, tests, documentation, CI
- ✓ **Autonomy** - You are 4, organize yourselves as you wish
- ✓ **Thoughtfulness** - Justify your technical choices
- ✓ **Flawless demo** - This is your moment to shine

What We Don't Want to See

- ✓ Untested code or code that doesn't compile
- ✓ Application that crashes during demo
- ✓ Disorganized team where one person did everything
- ✓ Excuses like ”we didn't have time”
- ✓ Improvised presentation

v 1.1.1

{EPITECH}